GUÍA PRÁCTICA

Diseño seguro de aplicaciones seguras, para gestionar adecuadamente la autenticación, autorización y control de Acceso.

1. Datos Generales

Carrera:	**Tecnología Superior Universitaria en Desarrollo de Software**
Período académico:	**30-2026-2P**
Asignatura:	**Programación Orientada a Objetos II (TERCERO)**
Unidad Nº:	**4**
Tema:	
Ciclo-Paralelo:	**M6A**
Práctica Nº:	**3**
Horas:	**5**
Estudiante:	**David Lopez** **Anthony Fajardo** **Stalyn Pesantez**

2. Contenido

2.1 Fundamentos

El presente proyecto se basa en los principios de la Programación Orientada a Objetos (POO), aplicando una arquitectura modular que separa las responsabilidades en tres capas principales:

- Modelo (Model): Maneja la lógica de datos y la conexión con la base de datos SQLite.
- Vista (View): Implementa la interfaz gráfica mediante PyQt6.
- Controlador (Controller): Actúa como intermediario entre la vista y el modelo.

2.2 Objetivo de la Guía

Desarrollar una aplicación de escritorio funcional que permita gestionar una biblioteca, integrando:

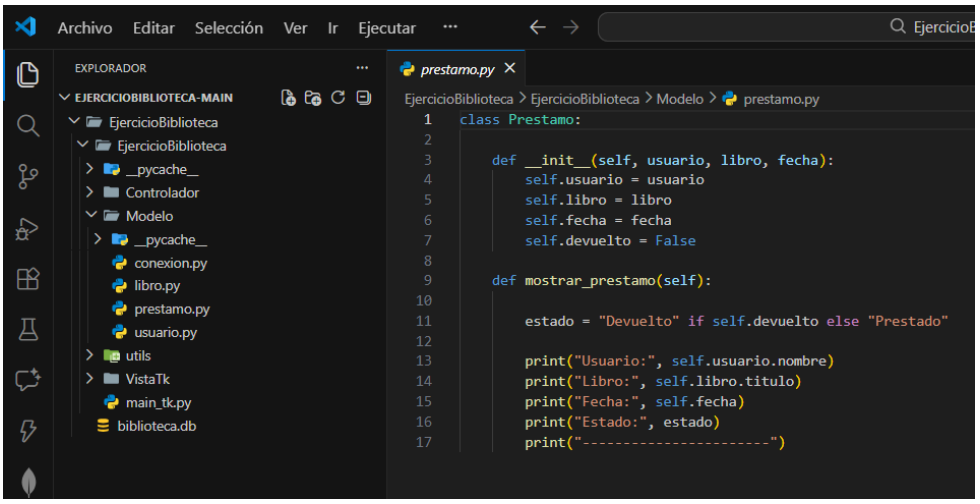
- Registro de libros y usuarios.
- Gestión de préstamos y devoluciones.
- Visualización de datos en tiempo real.
- Uso de base de datos relacional.
- Interfaz gráfica moderna e intuitiva.

2.3 Procedimiento a emplear

El sistema fue desarrollado bajo una arquitectura modular basada en el patrón Modelo-Vista-Controlador (MVC), lo que permite una adecuada separación de responsabilidades dentro de la aplicación. Esta estructura facilita la organización del código, donde el modelo se encarga de la gestión de datos, la vista de la interacción con el usuario mediante la interfaz gráfica, y el controlador de la lógica del sistema y la comunicación entre ambos.

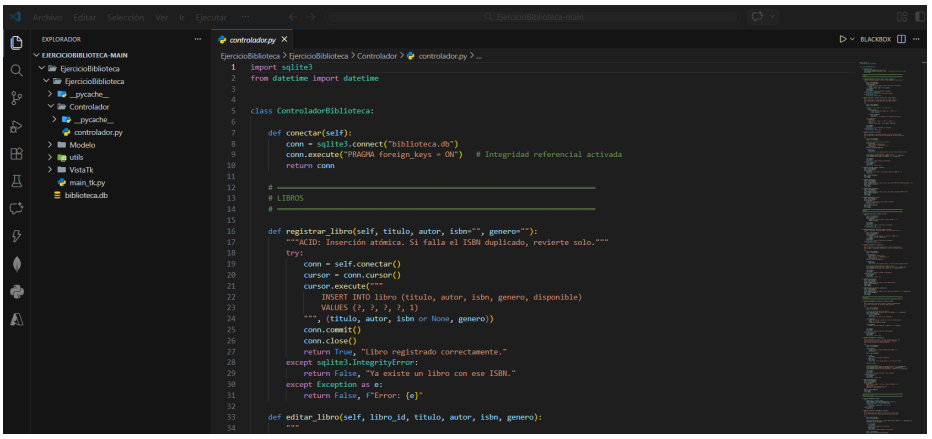
1. Implementación del modelo

   En la implementación del modelo se desarrolló la conexión a la base de datos SQLite, permitiendo la comunicación entre la aplicación y el almacenamiento de datos. Se crearon funciones encargadas de generar automáticamente las tablas necesarias en caso de que no existan, asegurando la inicialización correcta del sistema.



2. Desarrollo del controlador

   El controlador fue diseñado para gestionar la lógica del sistema, implementando métodos que permiten realizar operaciones como registrar usuarios, libros, gestionar préstamos y devoluciones. Además, se encargó de controlar el flujo de datos entre la vista y la base de datos, asegurando que la información se procese correctamente.

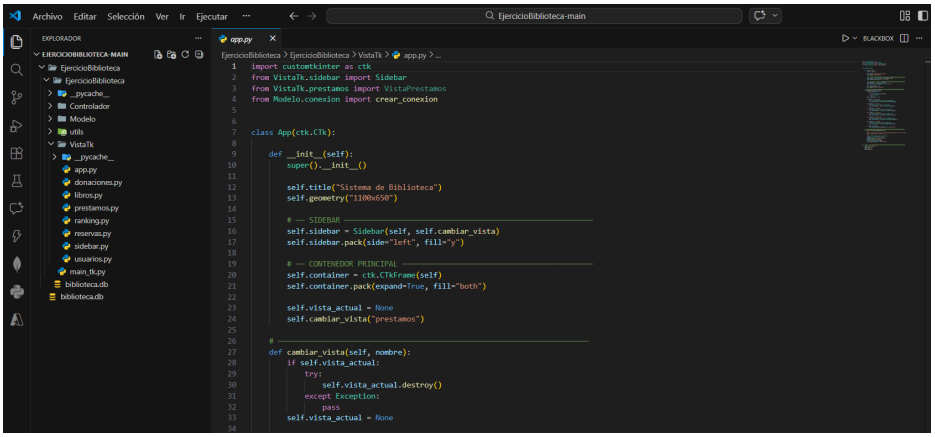


```python
import sqlite3
from datetime import datetime


class ControladorBiblioteca:

    def conectar(self):
        conn = sqlite3.connect("biblioteca.db")
        conn.execute("PRAGMA foreign_keys = ON")  # Integridad referencial activada
        return conn

    # ═══════════════════════════════════════════════════════════
    # LIBROS
    # ═══════════════════════════════════════════════════════════

    def registrar_libro(self, titulo, autor, isbn="", genero=""):
        """ACID: Inserción atómica. Si falla el ISBN duplicado, revierte solo."""
        try:
            conn = self.conectar()
            cursor = conn.cursor()
            cursor.execute("""
                INSERT INTO libro (titulo, autor, isbn, genero, disponible)
                VALUES (?, ?, ?, ?, 1)
            """, (titulo, autor, isbn or None, genero))
            conn.commit()
            conn.close()
            return True, "Libro registrado correctamente."
        except sqlite3.IntegrityError:
            return False, "Ya existe un libro con ese ISBN."
        except Exception as e:
            return False, f"Error: {e}"

    def editar_libro(self, libro_id, titulo, autor, isbn, genero):
        """
```

3. Construcción de la interfaz gráfica

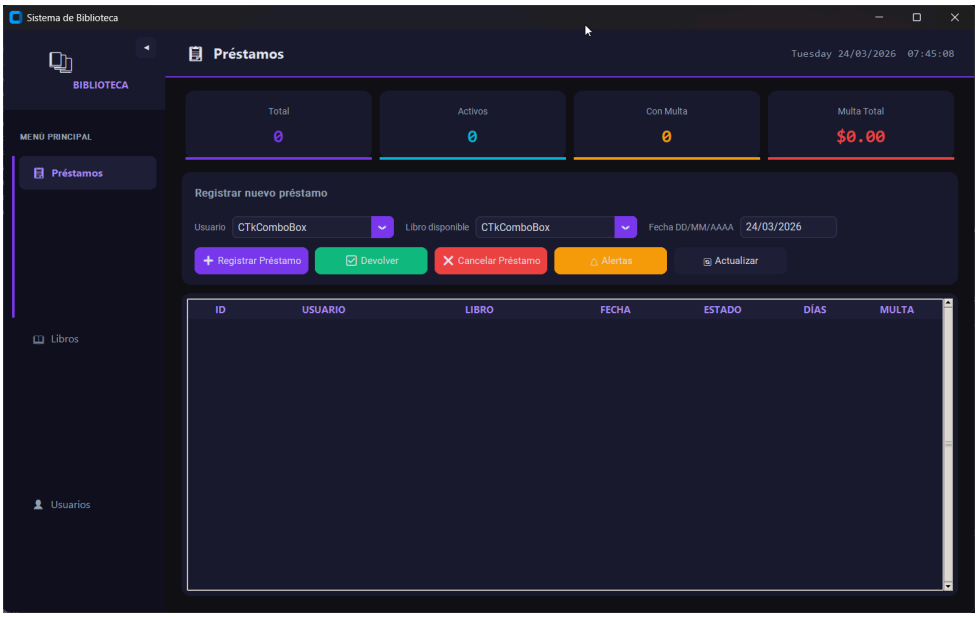
La interfaz gráfica fue desarrollada utilizando la librería PyQt6, permitiendo crear un entorno visual moderno, interactivo y fácil de usar. Se diseñaron formularios para el registro de usuarios y libros, así como para la gestión de préstamos. Además, se implementó una tabla que permite visualizar la información en tiempo real, mostrando datos como usuario, libro, fecha y estado

La estructura de las vistas:

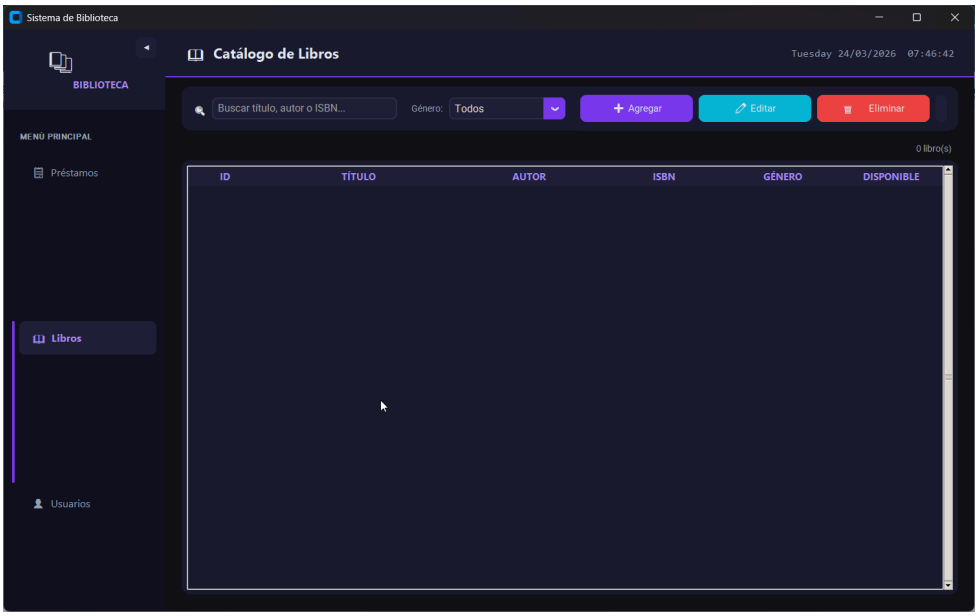


```python
import customtkinter as ctk
from VistaTk.sidebar import Sidebar
from VistaTk.prestamos import VistaPrestamos
from Modelo.conexion import crear_conexion


class App(ctk.CTk):

    def __init__(self):
        super().__init__()

        self.title("Sistema de Biblioteca")
        self.geometry("1100x650")

        # ── SIDEBAR ──────────────────────────────────────────
        self.sidebar = Sidebar(self, self.cambiar_vista)
        self.sidebar.pack(side="left", fill="y")

        # ── CONTENEDOR PRINCIPAL ─────────────────────────────
        self.container = ctk.CTkFrame(self)
        self.container.pack(expand=True, fill="both")

        self.vista_actual = None
        self.cambiar_vista("prestamos")

    # ═══════════════════════════════════════════════════════════
    def cambiar_vista(self, nombre):
        if self.vista_actual:
            try:
                self.vista_actual.destroy()
            except Exception:
                pass
        self.vista_actual = None
```

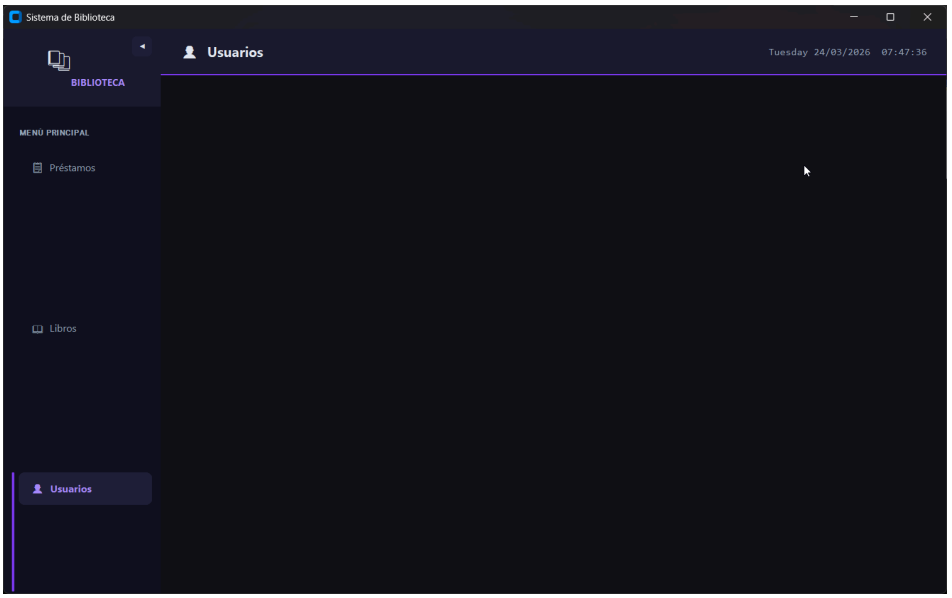
Préstamos:



Libros:



Usuarios:



Generar instalador de la aplicación

1)Descargamos la librería de PyInstaller con el comando

pip install pyinstaller

2) Generar el archivo ejecutable (.exe)

Ubícate en la carpeta de tu proyecto y ejecuta:

pyinstaller --onefile --windowed main.py

Explicación:

- `--onefile` → genera un solo archivo `.exe`
- `--windowed` → evita que se abra la consola (ideal para interfaces gráficas)

Se crearán estas carpetas:

- `build/`
- `dist/` → aquí está tu ejecutable

3) Verificar el ejecutable

Entra en la carpeta:

dist/

Y ejecuta tu archivo:

InstaladorBiblioteca.exe

Librerías de pruebas de código

1) Instalamos la librería pytest

pip install pytest

¿Cómo funciona?

- Detecta automáticamente archivos que empiecen con `test_`
- Ejecuta funciones que también empiecen con `test_`
- Usa `assert` para verificar resultados
- Si la condición es verdadera → pasa
- Si falla → muestra el error

2) Usamos la librería unittest (incluida en Python)

¿Cómo funciona?

- Se basa en clases que heredan de `TestCase`
- Cada prueba es un método dentro de la clase
- Usa métodos como:
  - `assertEqual()`
  - `assertTrue()`
- Ejecuta todas las pruebas al correr el archivo

3) Instalamos la librería nose2

pip install nose2

¿Cómo funciona?

- Detecta automáticamente archivos y funciones de prueba
- Es compatible con `unittest`
- Ejecuta pruebas sin necesidad de mucha configuración
- Muestra resultados en consola (éxito o fallo)

Resumen rápido

- **pytest** → automático y simple
- **unittest** → estructurado por clases
- **nose2** → automático + compatible con unittest

2.4 Resultados esperados

Resultados esperados

La interfaz gráfica fue desarrollada utilizando la librería PyQt6, permitiendo crear un entorno visual moderno, interactivo y fácil de usar. Se diseñaron formularios para el registro de usuarios y libros, así como para la gestión de préstamos. Además, se implementó una tabla que permite visualizar la información en tiempo real, mostrando datos como usuario, libro, fecha y estado.